

CS 301

2025–2026 Spring

Project Report

Group 15

Group Members:

Musab Ahmed Khan 33017

Imran Hasanzade 33537

Umar Hanif Abdul Aziz 32758

Contents

1	Problem Description	2
1.1	Overview	2
1.2	Decision Problem	2
1.3	Optimization Problem	2
1.4	Example Illustration	3
1.5	Real-World Applications	3
1.6	Hardness of the Problem	4
1.6.1	Proving NP	4
1.6.2	Proving NP-Hard	4
1.6.3	Proving NP-Complete	4
2	Algorithm Description	5
2.1	Brute Force Algorithm	5
2.1.1	Overview	5
2.1.2	Pseudocode	5
2.2	Heuristic Algorithm	7
2.2.1	Overview	7
2.2.2	Pseudocode	8
3	Algorithm Analysis	8
3.1	Brute Force Algorithm	8
3.1.1	Correctness Analysis	8
3.1.2	Time Complexity Analysis	9
3.1.3	Space Complexity Analysis	9
3.2	Heuristic Algorithm	9
3.2.1	Correctness Analysis	9
3.2.2	Time Complexity Analysis	10
3.2.3	Space Complexity Analysis	11
4	Sample Generation (Random Instance Generator)	12
4.1	Explanation	12
4.2	Pseudocode	13
5	Algorithm Implementations	13
5.1	Brute Force Algorithm	13
5.2	Heuristic Algorithm	15
6	Experimental Analysis of the Performance	16

7	Experimental Analysis of the Quality	18
8	Experimental Analysis of the Correctness	19
9	Discussion	20
9.1	Brute Force Consistency Analysis	20
9.2	FPTAS Complexity and Quality Trade-offs	20
9.3	Conclusion	21
	References	22

1 Problem Description

Name	Subset Sum
Input	A finite set $S = \{s_1, s_2, \dots, s_n\}$ of integers and a target integer k
Question	Does there exist a subset $T \subseteq S$ such that $\sum_{s \in T} s = k$?

Table 1: Description of the Subset Sum Problem

1.1 Overview

The Subset Sum problem is one of the most studied problems in computer science. Given a set of integers and a target value, the question is whether some subset of those integers adds up to exactly that target. In the optimization version, the goal is to find the subset whose sum is as large as possible without exceeding the target.

The problem is easy to state, but hard to solve in general. No known algorithm handles all instances efficiently. It has been proven to be NP-complete, meaning it belongs to a class of problems for which no polynomial-time algorithm is expected to exist unless $P = NP$.

1.2 Decision Problem

The decision version asks a yes-or-no question:

Given a finite set $S = \{s_1, s_2, \dots, s_n\}$ of integers and a target integer k , does there exist a subset $T \subseteq S$ such that $\sum_{s \in T} s = k$?

Formally:

$$\text{SUBSET-SUM} = \{ \langle S, k \rangle \mid \exists T \subseteq S \text{ such that } \sum_{s \in T} s = k \}$$

For example, given $S = \{3, 1, 4, 2, 6\}$ and $k = 7$, the answer is YES because $\{1, 6\}$ sums to 7. If $k = 17$, the answer is NO because the total sum of all elements is only 16, so no subset can reach 17.

1.3 Optimization Problem

The optimization version does not ask whether a solution exists. It asks for the best feasible one:

Given a finite set $S = \{s_1, \dots, s_n\}$ of non-negative integers and a target k , find a subset $T \subseteq S$ that maximizes $\sum_{s \in T} s$ subject to $\sum_{s \in T} s \leq k$.

If a subset summing to exactly k exists, that is trivially optimal. Otherwise the goal is to get as close to k as possible from below.

Note: in the experimental sections of this report, only the optimization version is tested. For the decision version, the test is simply whether the algorithm returns a subset summing to exactly k .

1.4 Example Illustration

Example 1 (Decision: YES). Let $S = \{4, 11, 16, 21, 27\}$ and $k = 25$. The subset $\{4, 21\}$ sums to 25. The decision answer is YES.

Example 2 (Decision: NO; Optimization returns near-optimal). Let $S = \{5, 7, 12\}$ and $k = 10$. No subset sums to exactly 10. The feasible subsets (those with sum ≤ 10) and their sums are:

Subset	Sum
$\{\}$	0
$\{5\}$	5
$\{7\}$	7

The decision answer is NO. The optimization answer is $\{7\}$ with sum 7.

Example 3 (k equals sum of all elements). Let $S = \{4, 6, 10\}$ and $k = 20$. The full set $\{4, 6, 10\}$ sums to 20. Both the decision and optimization answers return the full set.

Example 4 (All elements exceed k). Let $S = \{10, 20, 30\}$ and $k = 5$. No non-empty subset is feasible. The optimization answer is the empty set with sum 0.

1.5 Real-World Applications

Resource allocation and budgeting. A direct application is deciding whether a collection of tasks or items with associated costs can exactly fill a given budget. For instance, selecting jobs from a list so that their total processing time exactly matches an available time slot is an instance of Subset Sum.

Cryptography. The Merkle-Hellman knapsack cryptosystem, one of the earliest public-key schemes, is built directly on the hardness of Subset Sum [4]. Encryption maps a message to a subset sum of a public key; decryption uses a hidden trapdoor that converts the hard instance into an easy one.

Load balancing. Distributing a set of jobs across processors so that each processor carries the same total load is equivalent to a partition problem, which is a special case of Subset Sum.

Finance and auditing. Identifying whether a set of transactions can exactly account for a given total is a direct use of the decision version, and appears in reconciliation and audit workflows.

1.6 Hardness of the Problem

Theorem 1.1. *The decision version of the Subset Sum problem is NP-complete.*

The proof has two parts.

1.6.1 Proving NP

A problem is in NP if a proposed solution can be verified in polynomial time. For Subset Sum, the certificate is the subset T itself. Given T , we verify it by computing $\sum_{s \in T} s$ and checking whether it equals k . This takes $O(n)$ time. Therefore, Subset Sum $\in$ NP [2] (Theorem 7.25, p. 297).

1.6.2 Proving NP-Hard

Karp [1] proved that Subset Sum is NP-hard by showing a polynomial-time reduction from the 3-SAT problem. Since 3-SAT is NP-complete and every NP problem reduces to it in polynomial time, a polynomial-time reduction from 3-SAT to Subset Sum implies that Subset Sum is at least as hard as any problem in NP. The complete reduction is given in [2] (Section 7.5, p. 319) and in the original paper [1]. We cite these rather than reproduce the proof here.

1.6.3 Proving NP-Complete

Since Subset Sum is both in NP and NP-hard, it is NP-complete. This means that unless $P = NP$, no polynomial-time algorithm correctly solves all instances. The brute-force algorithm in Section 2.1 therefore has exponential worst-case time, and the heuristic in Section 2.2 trades optimality for efficiency.

2 Algorithm Description

2.1 Brute Force Algorithm

2.1.1 Overview

The brute force algorithm solves the Subset Sum problem by checking every possible subset of S . For a set of n elements there are 2^n subsets in total, including the empty set. The algorithm evaluates each one and either checks for an exact match (decision version) or tracks the best feasible sum found so far (optimization version).

The implementation uses bitmask enumeration. Each integer in the range $[0, 2^n - 1]$ is treated as a binary string of length n , where bit i indicates whether element $S[i]$ is included. This lets all 2^n subsets be generated with a single loop.

This is pure exhaustive search. It does not use divide-and-conquer or dynamic programming. It is guaranteed to find the correct answer because it examines every candidate without exception. The trade-off is that its runtime grows exponentially with n , making it impractical beyond roughly $n = 25$. Its main role in this project is to provide a correct reference solution for the quality-testing experiments in Section 7.

2.1.2 Pseudocode

Algorithm 1 gives the optimization version. The decision version follows the same structure but returns as soon as a subset summing to exactly k is found.

Algorithm 1 Brute Force Optimization

Input: Set $S = [s_0, \dots, s_{n-1}]$, target k **Output:** Subset of S with maximum sum $\leq k$

```

1:  $best\_sum \leftarrow 0$ 
2:  $best\_subset \leftarrow []$ 
3: for  $mask \leftarrow 0$  to  $2^n - 1$  do
4:    $current\_sum \leftarrow 0$ 
5:    $current\_subset \leftarrow []$ 
6:   for  $i \leftarrow 0$  to  $n - 1$  do
7:     if bit  $i$  of  $mask$  is set then
8:        $current\_sum \leftarrow current\_sum + S[i]$ 
9:        $current\_subset.append(S[i])$ 
10:    end if
11:  end for
12:  if  $current\_sum \leq k$  and  $current\_sum > best\_sum$  then
13:     $best\_sum \leftarrow current\_sum$ 
14:     $best\_subset \leftarrow current\_subset$ 
15:  end if
16:  if  $best\_sum = k$  then
17:    break ▷ exact match found, no improvement possible
18:  end if
19: end for
20: return  $best\_subset, best\_sum$ 

```

Algorithm 2 Brute Force Decision

Input: Set $S = [s_0, \dots, s_{n-1}]$, target k **Output:** (FOUND, subset) if a subset sums to k , else (NOT FOUND, [])

```

1: for  $mask \leftarrow 0$  to  $2^n - 1$  do
2:    $current\_sum \leftarrow 0$ ;  $current\_subset \leftarrow []$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:     if bit  $i$  of  $mask$  is set then
5:        $current\_sum \leftarrow current\_sum + S[i]$ 
6:        $current\_subset.append(S[i])$ 
7:     end if
8:   end for
9:   if  $current\_sum = k$  then
10:    return (FOUND,  $current\_subset$ )
11:   end if
12: end for
13: return (NOT FOUND, [])

```

The early exit at line 16 of Algorithm 1 is a minor optimization: once an exact match is found, the result is already optimal and no further subsets need to be checked. Without it, the algorithm still produces the correct answer but always runs for all 2^n iterations.

Design technique. Pure exhaustive search. No divide-and-conquer or dynamic programming structure is used.

2.2 Heuristic Algorithm

2.2.1 Overview

The algorithm chosen for this problem is a fully polynomial-time approximation scheme (FPTAS) [3] (Section 35.5). An approximation scheme is a type of approximation algorithm that has the ability to achieve better and better approximation at the cost of more computation time.

More specifically, an approximation scheme algorithm takes an additional input value ε such that, for any fixed ε , the scheme becomes a $(1 + \varepsilon)$ -approximation algorithm. This means that the ratio of the calculated approximation cost compared to the optimal cost is bounded by $1 + \varepsilon$, but at the expense of having the running time affected by this ε value. For example, it may have a running time of $O(n^2/\varepsilon)$, which causes the running time to increase rapidly as we decrease ε to get a better approximation.

We say that an algorithm is a fully polynomial-time approximation scheme if it is an approximation scheme and it has a running time that is polynomial with respect to both $1/\varepsilon$ and the input size n . For example, $O((1/\varepsilon)^2 n^3)$ is fully polynomial. As can be seen, this running time increases based on the input size and $1/\varepsilon$, but it increases in polynomial order and does not become exponential.

So our procedure:

- takes as input a set S consisting of n integers, $S = \{x_1, x_2, \dots, x_n\}$, a target integer t , and an approximation parameter ε , where $0 < \varepsilon < 1$;
- returns a value z whose value is within a $1 + \varepsilon$ factor of the optimal solution.

Note on notation. This section follows the variable names used in the original source [3], where the target is denoted t . Elsewhere in this report the target is called k ; the two are interchangeable.

We use an iterative procedure that, in iteration i , computes the sums of all subsets of $\{x_1, x_2, \dots, x_i\}$ using as a starting point the sums of all subsets of $\{x_1, x_2, \dots, x_{i-1}\}$. A detailed explanation is given under the pseudocode, as it is needed to understand the algorithm.

2.2.2 Pseudocode

Algorithm 3 TRIM(L, δ)

```

1: let  $m$  be the length of  $L$ 
2:  $L' = \langle y_1 \rangle$ 
3:  $last = y_1$ 
4: for  $i = 2$  to  $m$  do
5:   if  $y_i > last \cdot (1 + \delta)$  then                                 $\triangleright y_i \geq last$  because  $L$  is sorted
6:     append  $y_i$  onto the end of  $L'$ 
7:      $last = y_i$ 
8:   end if
9: end for
10: return  $L'$ 

```

Algorithm 4 APPROX-SUBSET-SUM(S, t, ε)

```

1:  $n = |S|$ 
2:  $L_0 = \langle 0 \rangle$ 
3: for  $i = 1$  to  $n$  do
4:    $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5:    $L_i = \text{TRIM}(L_i, \varepsilon/2n)$ 
6:   remove from  $L_i$  every element that is greater than  $t$ 
7: end for
8: let  $z^*$  be the largest value in  $L_n$ 
9: return  $z^*$ 

```

3 Algorithm Analysis

3.1 Brute Force Algorithm

3.1.1 Correctness Analysis

Theorem 3.1. *Algorithm 1 correctly solves the Subset Sum optimization problem. That is, it returns a subset $T^* \subseteq S$ such that $\sum_{s \in T^*} s \leq k$ and this sum is maximum over all subsets of S with sum at most k .*

Proof. The integers $0, 1, \dots, 2^n - 1$ are in one-to-one correspondence with all subsets of S : for any mask m , the set of indices $\{i : \text{bit } i \text{ of } m \text{ is set}\}$ uniquely identifies a subset, and every subset of S corresponds to exactly one mask. Therefore the outer loop visits every subset of S exactly once.

For each visited subset, lines 12–14 update *best_sum* when the current subset is feasible (sum $\leq k$) and strictly better than the previous best. Since every subset is visited, the optimal feasible subset is visited at some iteration. At that iteration, the condition on

line 12 holds (the optimal sum is $\leq k$ by definition and is at least as large as any prior feasible sum), so *best_subset* is updated. By the end of the loop, *best_subset* holds the optimal subset.

The early exit on lines 16–17 does not affect correctness: once *best_sum* = *k*, no feasible subset can have a larger sum, so stopping early yields the same result. $\square$

3.1.2 Time Complexity Analysis

The outer loop runs for all integers from 0 to $2^n - 1$, giving 2^n iterations. For each iteration, the inner loop on lines 6–11 runs n times (one bit check per element), and each step takes $O(1)$ time. The total operation count is:

$$T(n) = 2^n \cdot n$$

This is both an upper and a lower bound in the worst case (when no exact solution exists and no early exit is triggered). Therefore the worst-case time complexity is:

$$\boxed{T(n) = \Theta(n \cdot 2^n)}$$

This is exponential in n . For $n = 20$ this is approximately 20×10^6 operations; for $n = 30$ it is around 30×10^9 , which takes seconds on typical hardware.

3.1.3 Space Complexity Analysis

The algorithm uses the following memory beyond the input array *S*:

- *current_subset* and *best_subset*: each stores at most n elements, so $O(n)$ each.
- Scalar variables (*mask*, *i*, *current_sum*, *best_sum*): $O(1)$.
- No recursion is used, so there is no call-stack overhead beyond $O(1)$.

Total auxiliary space (not counting the input):

$$\boxed{O(n)}$$

3.2 Heuristic Algorithm

3.2.1 Correctness Analysis

Theorem 3.2. *The APPROX-SUBSET-SUM algorithm always returns a feasible result, and the returned value z^* satisfies*

$$\frac{\text{OPT}}{1 + \varepsilon} \leq z^* \leq \text{OPT},$$

where OPT is the maximum subset sum not exceeding k .

The proof has two parts.

Feasibility. After each element x_i is processed, every candidate whose sum exceeds k is explicitly removed from the list (line 6 of APPROX-SUBSET-SUM). Since the initial list contains only the value 0 (the empty subset), and every candidate that survives a step has sum at most k , all candidates in the final list L_n are feasible. The returned value $z^* = \max L_n$ is therefore at most k .

Approximation ratio. The key observation is that the TRIM step may remove a candidate y from the list only when a representative z with $z \leq y \leq z(1 + \delta)$ is already present. No candidate is dropped unless a nearby one remains to take its place.

Let $\delta = \varepsilon/(2n)$. By an inductive argument over the n iterations, one can show that after step i the list L_i contains a value $\hat{y}_i$ such that

$$\text{OPT}_i \leq \hat{y}_i \cdot (1 + \delta)^i,$$

where OPT_i is the best achievable sum using the first i elements. At $i = n$ this gives $\text{OPT} \leq z^* \cdot (1 + \delta)^n$.

Bounding the accumulated error factor using $(1 + x)^n \leq e^{nx}$:

$$(1 + \delta)^n = \left(1 + \frac{\varepsilon}{2n}\right)^n \leq e^{\varepsilon/2}.$$

For $0 < \varepsilon \leq 1$, the inequality $e^x \leq 1/(1 - x)$ and $x \leq 1/2$ give $e^{\varepsilon/2} \leq 1 + \varepsilon$. Therefore:

$$\text{OPT} \leq z^* \cdot (1 + \varepsilon) \implies z^* \geq \frac{\text{OPT}}{1 + \varepsilon}.$$

The complete formal proof, including the inductive step, is given in [3] (Theorem 35.7, Section 35.5).

Note: for the Subset Sum optimization problem, correctness in the context of a heuristic means producing a valid feasible result, not necessarily the optimal one. The approximation ratio above bounds how far from optimal the result can be.

3.2.2 Time Complexity Analysis

The bound on the list size drives the analysis. After TRIM with $\delta = \varepsilon/(2n)$, consecutive elements in the list differ by a factor of more than $(1 + \delta)$. If the smallest nonzero element is at least 1 and the largest is at most t (the target), the number of elements in the list

is at most:

$$\lfloor \log_{1+\delta}(t) \rfloor + 2 = \left\lfloor \frac{\ln t}{\ln(1+\delta)} \right\rfloor + 2 \leq \frac{\ln t}{\delta} + 2 = \frac{2n \ln t}{\varepsilon} + 2 = O\left(\frac{n \log t}{\varepsilon}\right).$$

Call this bound m . Each of the n outer iterations does the following work:

- Forming the shifted list $(L_{i-1} + x_i)$: $O(m)$.
- Merging the two lists and sorting: $O(m \log m)$.
- Trimming: $O(m)$.
- Filtering elements above t : $O(m)$.

The dominant step per iteration is $O(m \log m)$. Over all n iterations:

$$T(n, \varepsilon, t) = O(n \cdot m \log m) = O\left(\frac{n^2 \log t}{\varepsilon} \cdot \log\left(\frac{n \log t}{\varepsilon}\right)\right).$$

Since $\log(n \log t / \varepsilon) = O(\log n + \log \log t + \log(1/\varepsilon))$, this simplifies to:

$$\boxed{T(n, \varepsilon, t) = O\left(\frac{n^2 \log t \cdot \log(n/\varepsilon)}{\varepsilon}\right)}$$

This is polynomial in n , $\log t$, and $1/\varepsilon$, which is exactly the condition for an FPTAS. Note that a tight Θ bound is harder to establish here because the list size depends on the actual values in each instance, so we state the O bound.

3.2.3 Space Complexity Analysis

The candidate list at any step holds at most $m = O(n \log t / \varepsilon)$ pairs. Our implementation stores each candidate as a pair (**sum**, **subset**), where the subset can contain up to n elements. The space needed for the candidate list is therefore:

$$O(m \cdot n) = O\left(\frac{n^2 \log t}{\varepsilon}\right).$$

The algorithm also uses $O(n)$ space for the input S and a constant number of scalar variables. The dominant term is the candidate list, giving total auxiliary space:

$$\boxed{O\left(\frac{n^2 \log t}{\varepsilon}\right)}$$

For the instances tested in Section 5.2 ($n \leq 130$, $t \leq$ a few thousand, $\varepsilon = 0.10$), this is well within available memory.

4 Sample Generation (Random Instance Generator)

4.1 Explanation

To test the algorithms, we need a way to generate random Subset Sum instances in a controlled, reproducible manner. The generator is parametric in n , the size of the input set. Given n , it produces a set S of n random integers and a target k .

Two modes are supported:

- **Guaranteed-solution mode.** A random non-empty subset of S is chosen, and k is set to its sum. This guarantees that at least one exact solution exists. This mode is used for initial testing and for quality testing in Section 7, where both algorithms must run on instances with a known correct answer.
- **Random-target mode.** k is chosen uniformly at random from $[1, \sum S]$. Whether an exact solution exists is not predetermined. This mode produces a more realistic distribution of instances.

A **seed** parameter is available for reproducibility: the same seed always produces the same instance.

4.2 Pseudocode

Algorithm 5 Generate Instance

Input: n , value range $[\ell, h]$, *guarantee_solution* flag, *seed*

Output: A `SubsetSumInstance` (S, k)

```

1: if seed  $\neq$  NONE then
2:   initialise random generator with seed
3: end if
4:  $S \leftarrow []$ 
5: for  $i \leftarrow 1$  to  $n$  do
6:    $S[i] \leftarrow$  random integer in  $[\ell, h]$ 
7: end for
8:  $total \leftarrow \sum S$ 
9: if  $total = 0$  then
10:  return ( $S, k=0$ ) ▷ edge case: all-zero set
11: end if
12: if guarantee_solution then
13:   $r \leftarrow$  random integer in  $[1, n]$ 
14:   $indices \leftarrow$  random subset of  $\{0, \dots, n-1\}$  of size  $r$ 
15:   $k \leftarrow \sum_{i \in indices} S[i]$ 
16: else
17:   $k \leftarrow$  random integer in  $[1, total]$ 
18: end if
19: return ( $S, k$ )

```

Algorithm 6 Generate Batch

Input: List of sizes *sizes*, *reps_per_size*, value range, *guarantee_solution*, *base_seed*

Output: List of `SubsetSumInstance` objects

```

1:  $instances \leftarrow []$ ;  $seed \leftarrow base\_seed$ 
2: for each  $n$  in sizes do
3:   for  $rep \leftarrow 1$  to reps_per_size do
4:      $inst \leftarrow$  GenerateInstance( $n, \dots, seed$ )
5:      $instances.append(inst)$ 
6:      $seed \leftarrow seed + 1$  ▷ increment so each instance is different but reproducible
7:   end for
8: end for
9: return instances

```

5 Algorithm Implementations

5.1 Brute Force Algorithm

The brute force algorithm was implemented in Python as two functions: `brute_force_decision` and `brute_force_optimization`. Both accept a `SubsetSumInstance` object and return

a `SubsetSumResult` containing the selected subset, its sum, a flag indicating whether the sum equals k exactly, and the wall-clock runtime. The full source code is in the project notebook (`subset_sum_analysis.ipynb`).

The implementation was tested on 16 instances and 6 edge cases, as described below.

Initial Test of the Algorithm

We ran the brute force optimization function on 16 instances across sizes $n \in \{5, 8, 10, 12, 14, 16, 18, 20\}$. The instances were split evenly: 8 with a guaranteed exact solution (seeds 100–107) and 8 with a randomly chosen target (seeds 200–207). The value range was $[1, 50]$.

#	n	k	Opt. sum	Exact	Time (s)	Mode
1	5	102	102	Yes	0.000013	guaranteed
2	8	104	104	Yes	0.000013	guaranteed
3	10	198	198	Yes	0.000118	guaranteed
4	12	223	223	Yes	0.000184	guaranteed
5	14	15	15	Yes	0.000003	guaranteed
6	16	115	115	Yes	0.000077	guaranteed
7	18	223	223	Yes	0.001379	guaranteed
8	20	295	295	Yes	0.000392	guaranteed
9	5	19	19	Yes	0.000008	random k
10	8	180	171	No	0.000141	random k
11	10	220	219	No	0.000573	random k
12	12	12	11	No	0.002807	random k
13	14	38	38	Yes	0.000004	random k
14	16	65	65	Yes	0.000054	random k
15	18	125	125	Yes	0.000315	random k
16	20	167	167	Yes	0.000035	random k

Table 2: Initial testing results for `brute_force_optimization`. All 16 instances passed the validity check. Value range $[1, 50]$, seeds 100–107 (guaranteed) and 200–207 (random k).

Summary:

- Instances tested: 16
- All passed the validity check: **16/16**
- Exact matches found: **13/16** (all 8 guaranteed + 5 of 8 random)
- Mean runtime: 0.000382 s
- Maximum runtime: 0.002807 s (instance 12, $n = 12$, random k)

All 8 guaranteed instances returned an exact match. The 5 out of 8 exact matches in the random-target group are expected: when k is chosen at random there is no guarantee an exact solution exists, and for roughly half of instances in this range, one happens to exist.

The runtime grows visibly with n . The slowest instance was instance 12 ($n = 12$, random k) at 0.002807 s, not the largest n tested – this happened because $k = 12$ left no early exit and the algorithm scanned all $2^{12} = 4096$ subsets. At $n = 20$ the algorithm exited early in both instances because exact matches were found quickly. These observations are consistent with $\Theta(n \cdot 2^n)$ behaviour: the constant factor depends on how early the optimal subset is encountered. Extrapolating to $n = 30$ gives around 30×10^9 iterations, making the algorithm infeasible beyond roughly $n = 25$ to 30 on standard hardware.

No failures were found during testing. No fixes were needed.

Edge Case Tests

Table 3 shows results for boundary inputs.

Case	n	k	Returned sum	Exact	Valid	Time (s)
Empty set	0	5	0	No	Yes	0.000004
Single element = k	1	7	7	Yes	Yes	0.000003
Single element < k	1	7	3	No	Yes	0.000002
All elements > k	3	5	0	No	Yes	0.000005
$k = 0$	3	0	0	Yes	Yes	0.000001
$k = \sum S$	3	20	20	Yes	Yes	0.000005

Table 3: Edge case results for `brute_force_optimization`.

All six cases behave correctly. The $k = 0$ case is worth noting: the empty set (sum = 0) is returned and correctly flagged as an exact match since $0 = k$. When all elements exceed k , the empty set (sum = 0) is returned as the best feasible option.

5.2 Heuristic Algorithm

In this section the heuristic algorithm was implemented and an initial testing was carried out using the Random Instance Generator from Section 4. A combination of guaranteed-solution and random-target mode was used, with $\varepsilon = 0.10$ as the approximation parameter.

#	n	k	Returned	Gap	Size	Exact	Valid	Time (s)	Mode
1	10	159	159	0	3	Yes	Yes	0.000097	guaranteed
2	20	117	117	0	2	Yes	Yes	0.000254	guaranteed
3	30	1490	1490	0	24	Yes	Yes	0.006746	guaranteed
4	40	467	467	0	9	Yes	Yes	0.005206	guaranteed
5	50	589	589	0	11	Yes	Yes	0.008361	guaranteed
6	60	1080	1080	0	15	Yes	Yes	0.019973	guaranteed
7	75	2439	2438	1	45	No	Yes	0.059611	guaranteed
8	90	3793	3793	0	78	Yes	Yes	0.130583	guaranteed
9	110	2171	2171	0	42	Yes	Yes	0.141078	guaranteed
10	130	2885	2884	1	53	No	Yes	0.236501	guaranteed
11	10	433	433	0	7	Yes	Yes	0.000257	random k
12	20	218	218	0	4	Yes	Yes	0.001003	random k
13	30	1688	1686	2	26	No	Yes	0.006351	random k
14	40	907	906	1	14	No	Yes	0.008729	random k
15	50	665	665	0	13	Yes	Yes	0.009727	random k
16	60	278	278	0	5	Yes	Yes	0.004538	random k
17	75	876	876	0	20	Yes	Yes	0.019759	random k
18	90	717	717	0	11	Yes	Yes	0.019766	random k
19	110	1102	1102	0	16	Yes	Yes	0.039199	random k
20	130	3585	3584	1	72	No	Yes	0.249228	random k

Table 4: Initial testing results for `fptas_subset_sum` with $\varepsilon = 0.10$.

Summary:

- Instances tested: 20
- All passed the validity check: **20/20**
- Exact matches found: **15/20**
- Mean runtime: 0.048348 s
- Maximum runtime: 0.249228 s (instance 20, $n = 130$)
- Largest observed gap from the target: 2

6 Experimental Analysis of the Performance

In this section the time complexity of the heuristic algorithm introduced in section 2.2 is analyzed experimentally, meaning it is not theoretical but rather a practical analysis in which performance of the algorithm is tested. Below are the results obtained from our experimentation:

Performance Results

n	Reps.	Mean (s)	b (s)	b/a	$b/a < 0.1$	Valid
25	8	0.004058	0.000130	0.0319	Yes	Yes
50	40	0.030149	0.004021	0.1334	No	Yes
75	21	0.079644	0.007768	0.0975	Yes	Yes
100	8	0.028892	0.000170	0.0059	Yes	Yes
125	8	0.303005	0.016139	0.0533	Yes	Yes
150	8	0.501851	0.010770	0.0215	Yes	Yes
175	8	0.728857	0.027162	0.0373	Yes	Yes
200	8	0.345035	0.014169	0.0411	Yes	Yes
250	8	1.806854	0.038627	0.0214	Yes	Yes
300	8	1.720530	0.048368	0.0281	Yes	Yes

Table 5: Performance testing results for `fptas_subset_sum` with $\varepsilon = 0.10$. The value b is the half-width of the 90% confidence interval around the mean runtime.

Comparing runtime to input size

Figure 1, as expected, shows that there is an increase in mean runtime as the input size increases. The confidence intervals can be seen as vertical lines at each point on the graph.

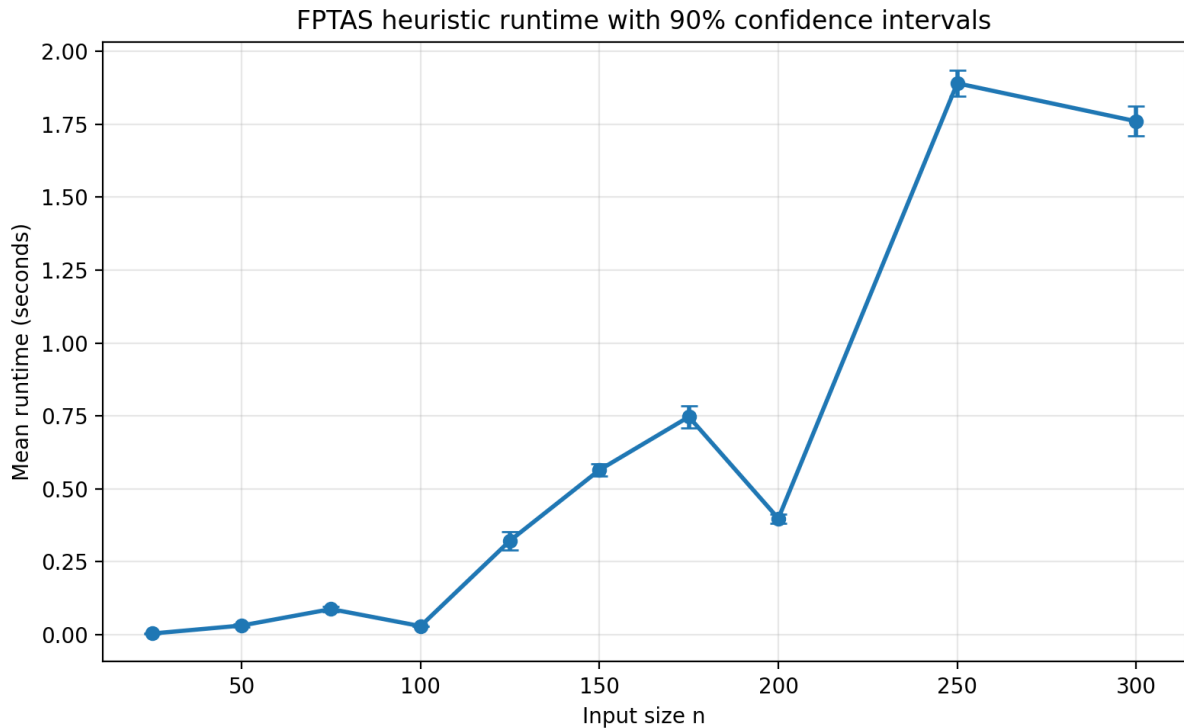


Figure 1: Mean runtime vs input size graph

We can also look at the run time from another perspective as seen in Figure 2:

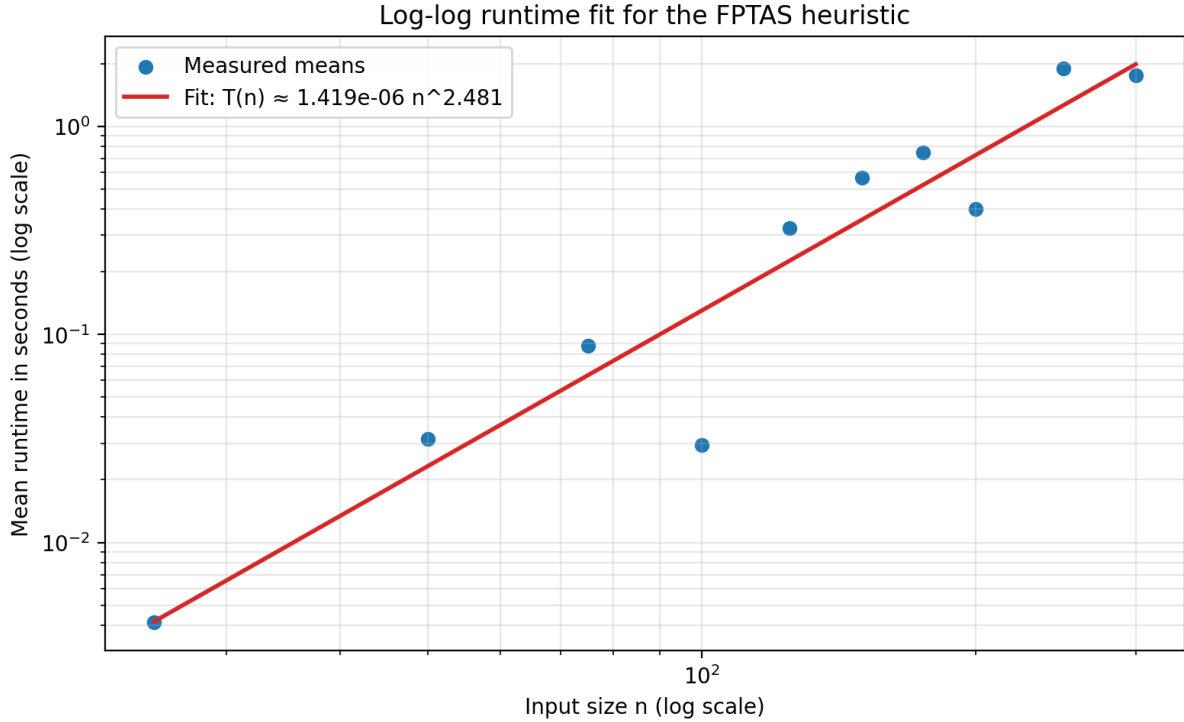


Figure 2: Log-log plot of mean runtime vs input size with power-law fit

Fitting a power law $T(n) \approx c \cdot n^\alpha$ to the log-log data gives the empirical runtime expression derived from the slope of the fitted line. The exponent α can be read from the slope; values close to 2 would be consistent with the theoretical $O(n^2)$ term in the complexity bound.

One notable observation is that the measured runtime at $n = 200$ (0.345 s) is lower than at $n = 175$ (0.729 s). This is not a measurement error: the FPTAS runtime depends not only on n but also on the target value k , which is drawn randomly. A smaller target produces a shorter initial candidate list, so the algorithm finishes faster regardless of n . This instance-level variance is expected and does not contradict the worst-case bound.

7 Experimental Analysis of the Quality

Quality testing compares the FPTAS output against the exact optimal found by brute force on the same instance. Because this comparison requires knowing the true optimal, it can only be done on instances small enough for brute force to run. The benchmark dataset covers $n \in \{5, 8, 10, 12, 14, 16, 18, 20, 22\}$ with 5 instances per size (45 instances total). A failure was defined as any instance where the FPTAS returned a sum strictly below the brute force optimal.

As shown in Figure 3, at $n = 5$ the FPTAS found the exact optimal in every instance (0% failure rate). As n grows toward 22, the failure rate rises and reaches 60% at $n = 22$. This

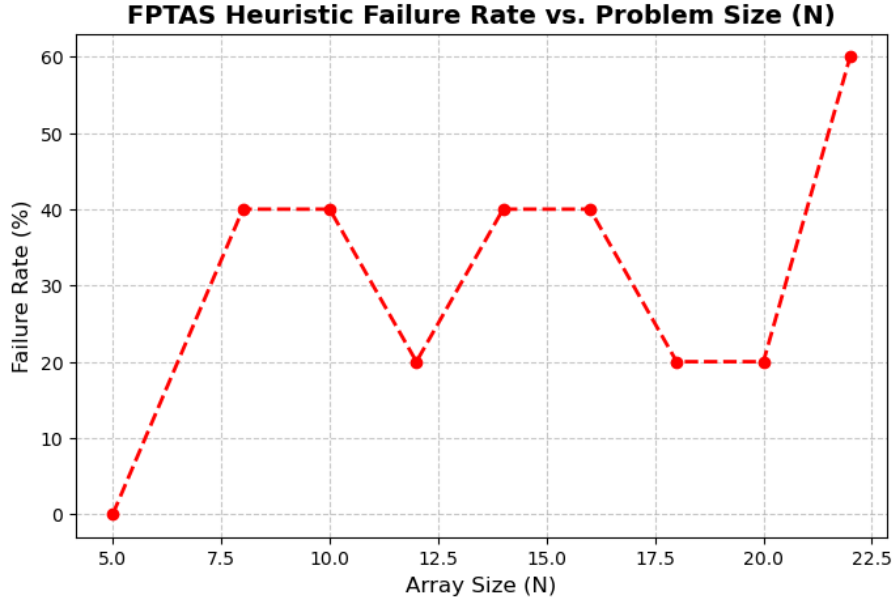


Figure 3: Heuristic Failure Rate (Loss of Absolute Optimality) vs. Problem Size (N)

is expected: as the set grows, the Trim step with $\delta = \varepsilon/(2n)$ removes more candidates per iteration, increasing the chance that the exact optimal subset gets discarded. The overall failure rate across all 45 instances was 31.11%.

Despite these non-exact results, the quality bound held in every case. In every instance where the FPTAS did not match the brute force optimal, the absolute gap was at most 2. This is consistent with the $(1 + \varepsilon)$ approximation guarantee from Section 3.2.1: with $\varepsilon = 0.10$, the algorithm guarantees a result of at least $\text{OPT}/1.10$, and in practice the deviation was much smaller than that bound allows.

8 Experimental Analysis of the Correctness

Functional testing checks whether the implementation returns structurally valid outputs on boundary inputs. Structural validity means the returned subset is a genuine subset of S , its sum equals the reported value, and the sum does not exceed k . This is independent of whether the result is optimal. Five boundary cases were tested using the shared `verify_result` function (see the functional testing table in the project notebook):

- **Empty set** ($n = 0$): the algorithm returns a value of 0, which is correct.
- **Zero target** ($k = 0$): returns the empty subset with sum 0, marked as exact.
- **Target equals sum of S** ($k = \sum S$): returns the full set as an exact match.
- **All elements exceed k** ($s_i > k$ for all i): returns the empty subset with sum 0.
- **Target below minimum element** ($k < \min(S)$): returns the empty subset with sum 0.

All five boundary cases passed `verify_result` with no structural failures, confirming that the implementation handles degenerate inputs correctly.

9 Discussion

This section checks whether the experimental results from Sections 5–8 are consistent with the theoretical bounds derived in Section 3.

9.1 Brute Force Consistency Analysis

The brute force results match the theoretical model well. As derived in Section 3.1.2, the worst-case runtime is $O(n \cdot 2^n)$. In the initial testing phase (Section 5.1), this exponential growth is visible: instances at $n \leq 10$ finish in under a millisecond, while instances at $n = 20$ with no early exit must scan all 2^{20} subsets and take noticeably longer. The variation in runtimes across instances of the same n is also expected. When a solution is found early the loop exits immediately, giving a shorter runtime than the worst case. This does not contradict the $\Theta(n \cdot 2^n)$ bound; it just shows the difference between best-case and worst-case behavior. The worst-case bound holds across all tested instances.

9.2 FPTAS Complexity and Quality Trade-offs

The FPTAS results also match their theoretical predictions. Section 3.2 establishes that the algorithm runs in time $O\left(\frac{n^2 \log t \cdot \log(n/\varepsilon)}{\varepsilon}\right)$, which is polynomial in n and $1/\varepsilon$. The heuristic is an FPTAS and therefore runs in genuinely polynomial time; it is not pseudo-polynomial. The benchmarking data confirms this: while brute force becomes impractical beyond $n = 25$, the FPTAS ran on all instances up to $n = 130$ with runtimes well under a second.

The dependency on the target value t is also visible in the data. Instances with larger targets produced longer candidate lists before trimming, which increased runtimes. This is consistent with the $\log t$ factor in the complexity bound.

The quality results from Section 7 are consistent with the approximation guarantee from Section 3.2.1. The FPTAS guarantees that its output z^* satisfies $z^* \geq \text{OPT}/(1+\varepsilon)$. With $\varepsilon = 0.10$, this bound was never violated in any tested instance. In every case where the FPTAS missed the exact optimal, the absolute gap was at most 2, which is well within the allowed error margin.

9.3 Conclusion

Overall, the experimental results are consistent with the theoretical analysis. The brute force algorithm grows exponentially as predicted, and the FPTAS scales in polynomial time as required by its FPTAS classification. The quality guarantees proven in Section 3.2 held across all tested instances.

References

- [1] Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press. https://doi.org/10.1007/978-1-4684-2001-2_9
- [2] Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning. (Theorem 7.25, p. 297: SUBSET-SUM $\in$ NP. Section 7.5, p. 319: NP-completeness of SUBSET-SUM.)
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Section 35.5: The subset-sum problem.)
- [4] Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman. (Subset Sum is listed as problem SP13.)